Compte Rendu semaine 30/03/2026

S’adapter à la piste fortmagma pour ne pas sortir du virage (CorrectionPiste)

Pour l’adaptation de notre agent à la piste FortMagma, nous avons travaillé sur l’amélioration de la fonction de correction latérale (CorrectionPiste), dont l’objectif est de maintenir le kart au centre de la piste, notamment dans les virages.

Nous avons choisi de ne pas modifier la logique de la fonction, mais plutôt de modifier ses paramètres de configuration pour ajuster le comportement du kart selon les différents types de virages et notamment le nombre de nœuds à regarder plus loin pour maintenir le kart au milieu de la piste. Or dans la map fortmagma, nous sortions sans cesse de la piste car le nombre de nœuds était à 2. La distance entre les nœuds peut être plus ou moins importante en fonction.

Le coefficient de correction a été fixé à 4.5 qui nous semble un bon compris d’après les tests et observations réalisées et un nombre de nœuds initialisé à 1.

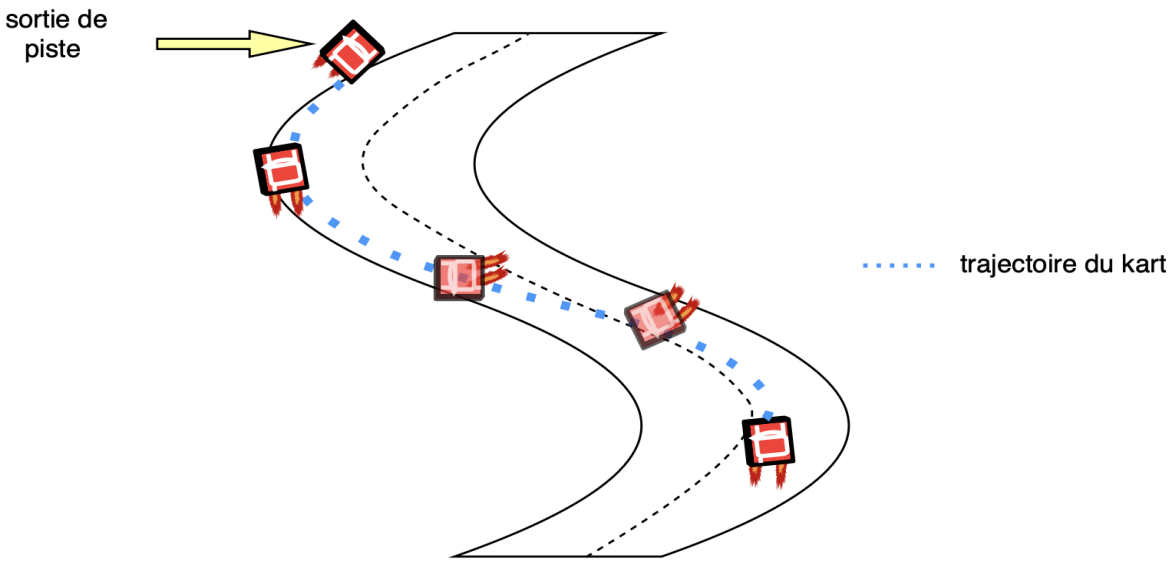
Notre objectif était d'avoir : Une adaptation fluide aux différents types de virages et une réduction des sorties de piste surtout dans les virages serrés.

Or pour la prochaine fois , nous nous chargerons de diminuer la trajectoire en zigzag à cause de nombreuses corrections apportées au final_steering (final_steering = item_steering+steering+correction_piste)

Permettre au kart de faire un drift - Code decideDrift

Pour implémenter ce code, nous avons rajouté deux conditions spécifiques afin d’activer le drift :

- **première condition :** lorsque l’angle du virage est important on active le drift et on évite d’être expulsé de la piste .
- **deuxième condition :** lorsqu’il n’y a pas de changement de direction (virage en forme de S). En présence d’un virage en forme de **S** et avec le drift activé , le kart sort de la piste. Pour cette raison, nous avons implémenté un petit code qui nous renvoie true (si changement de direction) ou false en fonction du virage rencontré pendant la course.


sortie de
piste
trajectoire du kart

implémenter un code qui cogne les adversaires proches de nous:

La classe HitRivals:
Son rôle est **d'attaquer les adversaires en fonçant sur eux** quand la situation est favorable.

1. Y a-t-il un item devant moi qui doit d'abord être évité ?
2. Suis-je dans un virage ou sur une ligne droite ?
3. Est-ce qu'un adversaire est proche pour attaquer ?

Initialisation de la classe:

AttackRivals : contient la logique pour viser et attaquer les karts adverses.
AnticipeKart : permet de détecter les virages et anticiper la trajectoire.
SteeringPiste : corrige la position du kart pour rester au centre de la piste.
Tout est basé sur la configuration cfg qui contient les paramètres du bot.

Détection d'un item:

```
def item_present(self, obs):
    items_pos = obs.get('items_position', [])

    for pos in items_pos:
        pos = np.array(pos)
        dist = np.linalg.norm(pos)

        # item devant et proche
        if pos[2] > 0 and dist < 20:
            return True

    return False
```

On récupère la liste des positions des items visibles (items_position).
Pour chaque item :

- On calcule sa distance par rapport à nous (np.linalg.norm(pos)).
- On regarde si l'item est devant (pos[2] > 0) et proche (dist < 20).

La logique d'attaque:

Pas d'item devant (not self.item_present(obs))
Pas de virage (detectVirage(obs) <= 0.2) → le bot préfère attaquer sur une ligne droite pour ne pas perdre le contrôle.

```
        karts_pos = obs.get('karts_position', [])
    pos = np.array(karts_pos[1])
    x, z = pos[0], pos[2]

    center_path = obs.get("center_path", [])
    centre = np.array(center_path[0])
    distance_centre = np.linalg.norm(pos - centre)

    #ne pas attaquer si adversaire trop éloigné du centre
    if distance_centre > cfg.hit.dist_centre:
        return None
```

On prend le kart proche de nous (index 1).
On calcule la distance entre l'adversaire et le centre de la piste.
Si le kart adverse est trop sur le côté, on ne l'attaque pas (return None)

```
    acceleration = 1
    target_angle = np.arctan2(x, z)
    nodes_path = obs.get("paths_start", [])

    if len(nodes_path) > 3:
        target_node = nodes_path[3]
        angle_target = np.arctan2(target_node[0], target_node[2])
        steering = np.clip(angle_target * 2, -1, 1)
    else:
        steering = 0

    correction_piste = self.steering.correction_centrePiste(obs)
    final_steering = np.clip(target_angle + correction_piste ,-1, 1)
```

target_angle : angle pour viser le kart adverse.
steering : angle pour suivre la trajectoire prévue par la piste.(noeuds)
correction_piste : ajustement pour rester au centre de la piste.
accélération : toujours 1 pour attaquer vite

Implémentation du drift

Dans le cadre du projet de course automatisée sur SuperTuxKart (STK), notre mission était d'implémenter une stratégie de drift pour l'agent DemoPilote. Le drift est une technique de pilotage qui permet au kart de glisser latéralement dans un virage afin de maintenir une trajectoire courbe à vitesse élevée.

L'objectif était double :

- Décider automatiquement quand activer le drift en fonction de la courbure de la piste et de la vitesse du kart.
- Adapter le steering pendant la glisse latérale pour éviter de sortir de la piste.

Le drift est contrôlé par le paramètre booléen `"drift"` dans le dictionnaire d'action retourné par `choose_action()`.

1- version actuelle

La solution finale repose sur une machine à 3 phases implémentée dans AccelerationControl.decideDrift(). Le schéma ci-dessous illustre les transitions entre phases :

Phase NORMAL

- Le kart surveille la courbure à chaque step via detectVirage().
- Le drift est déclenché si trois conditions sont simultanément remplies :
  1. courbure > cfg.virages.drift (seuil d'entrée)
  2. vitesse > cfg.drift.vitesse_min (vitesse minimale)
  3. changementDirection() == False (pas de changement de côté)

Phase DRIFTING — Maintien forcé

À l'entrée en drift, un compteur est initialisé à `cfg.drift.duree_min` steps. Pendant cette période, le drift est maintenu de force même si la courbure baisse légèrement. Deux annulations d'urgence existent :

- Changement de côté détecté (risque de tête-à-queue).
- Vitesse < vitesse_min × 0.6 (kart trop lent pour tenir la glisse).

Phase DRIFTING — Maintien conditionnel (après duree_min)

Après la période forcée, le drift se poursuit tant que la courbure reste au-dessus de `seuil_sortie = virages.drift × seuil_sortie_ratio`. Ce seuil de sortie, inférieur au seuil d'entrée, constitue l'hystérésis qui évite les désactivations prématurées.

Sortie propre

Quand les conditions de maintien ne sont plus remplies, _stop_drift() remet tous les attributs d'état à zéro et la machine repasse en phase NORMAL.

on a egalement tenté d'adapter le steering en fonction du drift:

1_steering_normal() — hors drift

En conduite normale, le steering est la somme de trois composantes :

```
final_steering = base_steering + correction_piste + item_steering
```

`base_steering` : cap vers le nœud anticipé (arctan2 du nœud path_lookahead devant).

`correction_piste` : correction de centrage issue de SteeringPiste, plafonnée à ±0.6.

`item_steering` : ajustement pour collecter les bons items ou esquiver les mauvais

2 _steering_drift() — pendant la glisse

Pendant le drift, appliquer la même formule est contre-productif : la correction de centrage s'oppose à la glisse latérale et l'annule. La fonction _steering_drift() remplace cette logique par :

```
correction_reduite  = correction_piste × cfg.drift.correction_ratio   # ex: × 0.2
poussee_interieure  = cote × cfg.drift.steer_interieur            # ex: +0.3 ou -0.3
final_steering      =    clip(base_steering    +    correction_reduite    + poussee_interieure, -1, 1)
```

correction_ratio = 0.2 Seuls 20 % de la correction de centrage sont appliqués. Le kart reste vaguement centré sans lutter contre la glisse.	**steer_interieur = 0.3** Le kart est légèrement poussé vers l'intérieur du virage (côté +1 ou -1), compensant la dérive latérale pour rester en piste.

Paramètres et optimisation Optuna

Tous les paramètres du drift sont centralisés dans configDemoPilote.yaml sous la clé drift: et intégrés à la fonction objective() d'optimise_kart.py. Optuna peut donc les optimiser automatiquement en cherchant à minimiser le nombre de steps total pour terminer les courses.